

RISC-V (RV32I) Processor Core

Oancea Alexandru

Microarchitecture Specification & Design Documentation

github.com/alex1oancea/riscv-pipelined-core

Table of Contents

RISC-V (RV32I) Processor Core	1
Chapter 1: Introduction	1
Chapter 2: Datapath Architecture	2
2.1. Program Counter (PC)	3
2.2. Register File	3
2.3. Arithmetic Logic Unit (ALU)	3
2.4. Multiplexers and Sign Extension	3
2.5. Critical Path Analysis	3
Chapter 3: Control Unit	4
3.1. Main Decoder	4
3.2. ALU Decoder	4
3.3. Hardware Design Considerations (Latch Prevention)	5
Chapter 4: Memory System and Integration	5
4.1. Instruction and Data Memories	5
4.2. Top-Level Integration	5
Chapter 5: Verification and Hardware Debugging	6
5.1. Self-Checking Testbench	6
5.1.2. Waveform Analysis	7

Chapter 1: Introduction

This document describes the architecture and implementation of a 32-bit processor based on the RISC-V (RV32I) instruction set. The project is developed in the SystemVerilog

hardware description language and serves an educational purpose, aiming for a deep understanding of how software instructions are translated into physical hardware signals.

The design and architecture of this processor are heavily based on the concepts and schematics presented in the reference book **"Digital Design and Computer Architecture, RISC-V Edition"** by Sarah L. Harris and David Harris.

The project is implemented iteratively: the initial version represents a Single-Cycle architecture (where each instruction is executed in a single clock cycle). Subsequently, the design will be scaled to a 5-stage Pipelined architecture (Fetch, Decode, Execute, Memory, Writeback), including hazard resolution mechanisms such as Forwarding and Stalling.

Chapter 2: Datapath Architecture

The Datapath represents the physical route that data and addresses take through the processor during the execution of an instruction. It consists of various interconnected functional units that operate based on the signals provided by the Control Unit.

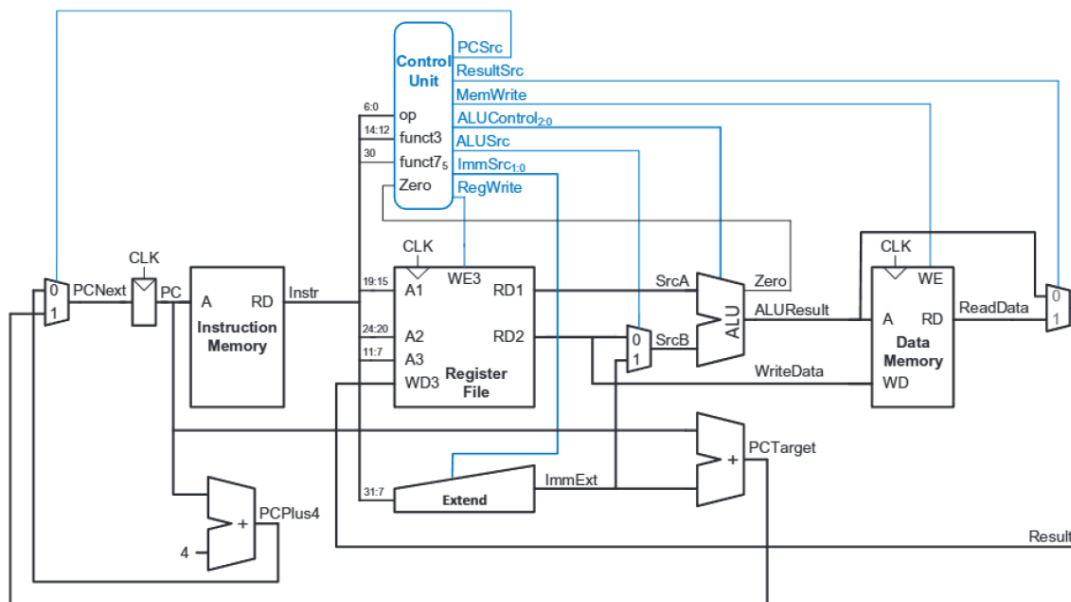


Figure 2.1: Complete single-cycle RISC-V microarchitecture. This schematic illustrates the datapath and control unit routing, serving as the physical blueprint for the SystemVerilog implementation. (Reference: Harris & Harris, Digital Design and Computer Architecture).

2.1. Program Counter (PC)

The PC is a 32-bit sequential register that holds the memory address of the current instruction being executed. On every rising edge of the clock (clk), the PC is updated to point to the next instruction. By default, it increments by 4 (since each instruction is 32 bits, or 4 bytes). In the case of branches or jumps, the PC is updated with a calculated target address.

2.2. Register File

The processor features a 32x32-bit Register File. It contains 32 general-purpose registers (x0 through x31), where x0 is hardwired to the constant value 0. The Register File allows two simultaneous asynchronous reads (outputs RD1 and RD2) and one synchronous write (input WD3, controlled by the RegWrite signal).

2.3. Arithmetic Logic Unit (ALU)

The ALU performs mathematical and bitwise operations. It receives two 32-bit operands (SrcA and SrcB) and a 3-bit control signal (ALUControl) from the ALU Decoder. It outputs a 32-bit ALUResult and a Zero flag. The Zero flag evaluates to 1 when the result is 0, which is critical for determining branch outcomes (e.g., beq).

2.4. Multiplexers and Sign Extension

To allow instructions to share datapath resources, multiplexers (MUX) act as physical switches controlled by the Control Unit:

- **ALUSrc MUX:** Selects the second operand for the ALU between a register value (RD2) and an immediate value.
- **ResultSrc MUX (MUX3):** Selects the final data to be written back to the Register File from either the ALU result, Memory Read Data, or PC+4.
- **Extend Module:** Takes immediate values of varying lengths from the instruction bits and sign-extends them to a full 32-bit representation.

2.5. Critical Path Analysis

In this single-cycle RISC-V microarchitecture, the critical path—which defines the maximum clock frequency (f_{max}) of the processor—is dictated by the Load Word (lw) instruction.

What makes lw uniquely demanding, and thus the slowest instruction in the datapath, is that it is the only instruction requiring three sequential accesses to major memory and state elements within a single clock cycle:

1. Reading the base address from the Register File.
2. Reading the data from the Data Memory.
3. Routing the data back to write it into the Register File.

Chapter 3: Control Unit

The Control Unit is the "brain" of the processor, responsible for decoding the instructions fetched from memory and generating the control signals that route data through multiplexers and dictate ALU operations. It is divided into two main sub-modules: the Main Decoder and the ALU Decoder.

3.1. Main Decoder

The mainDecoder module evaluates the 7-bit opcode of the instruction to determine its general category (memory operations, arithmetic operations, branches, etc.). Based on this opcode, the module generates the primary control signals for the Datapath.

Main Decoder Truth Table:

Instruction	Opcode	RegWrite	ImmSrc	ALUsrc	MemWrite	ResultSrc	Branch	ALUop	Jump
lw	0000011	1	00	1	0	01	0	00	0
sw	0100011	0	01	1	1	Xx	0	00	0
R-type	0110011	1	xx	0	0	00	0	10	0
beq	1100011	0	10	0	0	xx	1	01	0
I-type	0010011	1	00	1	0	00	0	10	0
jal	1101111	1	11	x	0	10	0	xx	1

(Note: "X" values represent "Don't Care" bits, indicating that the state of that specific signal does not affect the execution of the current instruction. This allows for hardware logic optimization during synthesis).

3.2. ALU Decoder

While the Main Decoder identifies the general instruction family, the aluDecoder module is responsible for specifying the exact mathematical or bitwise operation the ALU must execute. It combines the ALUOp signal received from the Main Decoder with the function bits from the instruction (funct3 and, in certain cases, bit 5 of funct7).

ALU Decoder Truth Table:

ALUop	Funct3	{op5, funct7[5]}	ALUcontrol	Instruction
00	x	x	000(add)	lw,sw
01	x	X	001(sub)	beq
10	000	x	000	add
	000	x	001	sub
	010	x	101	slt
	110	x	011	or
	111	x	010	and

3.3. Hardware Design Considerations (Latch Prevention)

In the SystemVerilog hardware description, both decoders were implemented using combinational logic (`always_comb`). To ensure the generation of a valid and stable logic circuit, and to prevent the synthesizer from unintentionally inferring memory elements (*Latches*), the decision blocks (case statements) strictly include a default: clause. This guarantees that for any unexpected or undefined combination of input signals, the outputs fall back to a predictable state, thereby maintaining the integrity of the datapath.

Chapter 4: Memory System and Integration

4.1. Instruction and Data Memories

The architecture follows a Harvard-like structure, separating the instruction and data memory paths to allow simultaneous fetching and data access in a single clock cycle.

- **Instruction Memory (`instrmem`):** Acts as a Read-Only Memory (ROM). It asynchronously outputs the 32-bit instruction located at the address provided by the Program Counter.
- **Data Memory (`datamem`):** Acts as a Random Access Memory (RAM). It handles Load (`lw`) and Store (`sw`) instructions. Reading is asynchronous, while writing is synchronous and strictly gated by the `MemWrite` control signal.

4.2. Top-Level Integration

The top module serves as the highest-level hardware wrapper. It instantiates the RISC-V processor core alongside the `instrmem` and `datamem`.

Chapter 5: Verification and Hardware Debugging

5.1. Self-Checking Testbench

To validate the processor's functionality, a self-checking testbench (tb.sv) was implemented. Instead of manually inspecting waveform signals for every clock cycle, the testbench automatically loads a machine-code program (riscvtest.txt) and monitors specific memory transactions. Upon completion, it asserts the final expected state.

Instruction	Desc.	Address	Machine Code
<i>main:</i> addi x2, x0, 5	# x2 = 5	0	00500113
addi x3, x0, 12	# x3 = 12	4	00C00193
addi x7, x3, -9	# x7 = (12 - 9) = 3	8	FF718393
or x4, x7, x2	# x4 = (3 OR 5) = 7	C	0023E233
and x5, x3, x4	# x5 = (12 AND 7) = 4	10	0041F2B3
add x5, x5, x4	# x5 = 4 + 7 = 11	14	004282B3
beq x5, x7, end	# shouldn't be taken	18	02728863
slt x4, x3, x4	# x4 = (12 < 7) = 0	1C	0041A233
beq x4, x0, around	# should be taken	20	00020463
addi x5, x0, 0	# shouldn't execute	24	00000293
<i>around:</i> slt x4, x7, x2	# x4 = (3 < 5) = 1	28	0023A233
add x7, x4, x5	# x7 = (1 + 11) = 12	2C	005203B3
sub x7, x7, x2	# x7 = (12 - 5) = 7	30	402383B3
sw x7, 84(x3)	# [96] = 7	34	0471AA23
lw x2, 96(x0)	# x2 = [96] = 7	38	06002103
add x9, x2, x5	# x9 = (7 + 11) = 18	3C	005104B3
jal x3, end	# jump to end, x3 = 0x44	40	008001EF
addi x2, x0, 1	# shouldn't execute	44	00100113
<i>end:</i> add x2, x2, x9	# x2 = (7 + 18) = 25	48	00910133
sw x2, 0x20(x3)	# [100] = 25	4C	0221A023
<i>done:</i> beq x2, x2, done	# infinite loop	50	00210063

5.1.2. Waveform Analysis

The following waveform validates the temporal behavior of the processor during simulation. Key signals such as the Program Counter (PC), fetched instruction (Instr), and ALU results are observable.

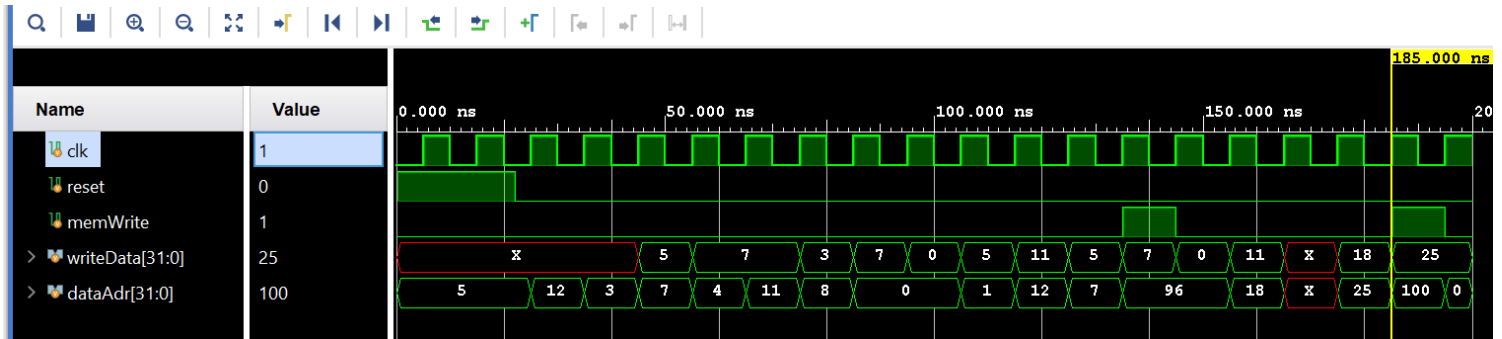


Figure 5.1: Vivado simulation waveform showing the successful execution of the test program. The cursor highlights the final clock cycles where MemWrite is asserted and the value 0x00000019 (25 in decimal) is driven on the WriteData bus to memory address 0x00000064 (100 in decimal).